ЗАДАЧА ДЛЯ ХАКАТОНА

Интеллектуальная система маршрутизации спецтехники (ИС УТО)

Оптимизация маршрутизации спецтехники на нефтяных месторождения

1. Контекст и описание проблемы

1.1. Бизнес-контекст

Компания эксплуатирует парк специализированной техники при выполнении производственных работ на нефтяных месторождениях. В рамках пилотного проекта рассматривается участок месторождения Жетыбай, где задействован парк подрядной организации численностью 52 единицы спецтехники различных типов. Планирование работ осуществляется на суточной основе с возможностью формирования горизонта планирования на несколько дней вперед. Производственные подразделения формируют заявки на использование техники. Старший диспетчер принимает заявки и передает их на распределение техники. Назначение техники на заявки выполняется вручную диспетчером автоколонны на основе опыта и приблизительной оценки расстояний. В течение одной рабочей смены единица техники может выполнять несколько заявок, перемещаясь между объектами работ.

Дорожная сеть включает как официальные дороги, так и накатанные полевые пути внутри месторождения, восстановленные из GPS-треков техники.

1.2. Проблема

Ручное планирование приводит к системным потерям по нескольким направлениям:

- Назначение техники на заявки выполняется диспетчером вручную на основе опыта и приблизительной оценки расстояний без автоматического расчета маршрута и времени прибытия (ETA).
- Планирование загрузки техники не учитывает автоматически длительность выполнения работ по заявкам, что усложняет оценку доступности техники в конкретный момент времени.
- Возможность выполнения нескольких заявок одной единицей техники за смену определяется диспетчером вручную и не оптимизируется системой.
- Приоритеты заявок обрабатываются неформально, без количественного учёта влияния на общую эффективность парка.

Последствия: В результате возникает увеличение холостого пробега техники, рост времени ожидания выполнения заявок, неравномерная загрузка парка и риск задержки выполнения срочных работ, что приводит к снижению общей эффективности использования автотранспорта и специальной техники.

2. Цель проекта

Разработать прототип интеллектуального сервиса, интегрируемого с системой КМГ, который автоматически рекомендует оптимальную единицу спецтехники для выполнения заявки на основе дорожного графа месторождения, текущего положения техники и параметров заявки. Сервис должен:

1. Рекомендовать оптимальную единицу спецтехники для выполнения заявки с обоснованием выбора, включая расстояние по графу дорог, расчетное время прибытия (ETA), приоритет заявки и доступность техники.
2. Строить маршруты по графу дорог месторождения (граф уже восстановлен из GPS-треков и предоставляется в готовом виде).
3. Учитывать приоритеты заявок: для высокого приоритета допускается выбор более далёкой, но свободной техники; для низкого допускается ожидание ближайшей.
4. Анализировать возможность выполнения нескольких заявок одной единицей техники в течение смены и оценивать целесообразность объединения таких заявок в один маршрут.
5. Отдавать результат через REST API и поддерживать визуализацию маршрутов (минимально — координатами/геометрией, на демо — на карте).

3. Формализация задачи оптимизации

3.1. Тип задачи

Задача представляет собой вариант Vehicle Routing Problem (VRP) со следующими особенностями:

- **Multi-Depot:** каждая единица техники стартует из своего текущего местоположения (множество виртуальных депо, а не одна центральная база).
- **Time Windows (VRPTW):** каждая заявка имеет временное окно выполнения, определяемое датой и сменой (дневная/ночная).
- **Multi-stop:** одна единица техники может последовательно выполнить несколько заявок в течение одной смены, если это оптимально с точки зрения времени движения и доступности техники.
- **Open-end routes:** после выполнения заявки техника не обязана возвращаться в исходную точку и может завершить маршрут в точке последней выполненной заявки.
- **Fleet compatibility:** не каждая единица техники может выполнять любую заявку. Совместимость определяется типом техники и типом выполняемых работ.

3.2. Целевая функция

Минимизировать суммарную стоимость обслуживания всех заявок, которая складывается из:

- общего пробега всех машин (суммарная длина маршрутов по графу дорог);
- штрафов за нарушение временных окон и приоритетов (soft constraints, высокоприоритетные заявки штрафуются сильнее за опоздание);
- суммарного времени простоя техники.

3.3. Ограничения

- Каждая заявка должна быть выполнена ровно одной единицей техники.
- Временные окна: исполнение заявки может превышать запланированное время на ее выполнение.
- Совместимость: техника назначается на заявку при совместимом типе работы.
- Маршрут строится строго по графу дорог (не по прямой).
- Техника остаётся на месте последней заявки (не возвращается на базу).

3.4. Свобода выбора подхода

Участники свободны в выборе методов решения. Возможные подходы включают, но не ограничиваются:

- **VRP-солверы**
- **Классические эвристики**.
- **Метаэвристики**.
- **AI/ML**.
- **Гибридные подходы**.

Приветствуется сравнение нескольких подходов и обоснование выбора.

4. Предоставляемые данные

Участники получают подключение к БД со следующими данными. Все координаты обезличены, но метрически согласованы между собой (расстояния между точками корректны).

4.1. Граф дорог: узлы (references.road_nodes)

Узлы дорожной сети месторождения, включая полевые дороги, восстановленные из GPS-треков. Каждый узел имеет уникальный идентификатор и координаты.

Поле	Тип	Описание
id	serial (PK)	Технический ключ
node_id	integer (unique)	Логический идентификатор узла для построения графа
lon	numeric(12,8)	Долгота (обезличена, метрически согласована)
lat	numeric(12,8)	Широта

Назначение: вершины графа дорог; привязка позиций техники и скважин к ближайшему узлу (map-matching).

4.2. Граф дорог: рёбра (references.road_edges)

Рёбра (дороги) между узлами графа. Каждое ребро имеет начальный узел, конечный узел и вес (стоимость проезда).

Поле	Тип	Описание
id	serial (PK)	Технический ключ
source	integer	ID начального узла (→ road_nodes.node_id)
target	integer	ID конечного узла (→ road_nodes.node_id)
weight	numeric(12,6)	Вес ребра

Связь: road_edges.source и road_edges.target ссылаются на road_nodes.node_id (логическая связь).

Назначение: построение графа для алгоритмов кратчайшего пути; расстояние маршрута = сумма weight рёбер пути.

4.3. Скважины / точки назначения (references.wells)

Справочник скважин и объектов работ. Координаты согласованы с графом дорог. Скважина является целевой точкой заявки: заявка содержит destination_uwi, по которому определяется местоположение.

Поле	Тип	Описание
id	serial (PK)	Технический ключ
uwi	varchar(50) (unique)	Уникальный идентификатор скважины
latitude	numeric(12,8)	Широта (обезличена)
longitude	numeric(12,8)	Долгота (обезличена)
well_name	varchar(255)	Отображаемое название скважины

Ограничение: (latitude, longitude) — либо обе заполнены, либо обе NULL.

4.4. Снапшоты техники (references.wialon_units_snapshot_1/2/3)

Три снапшота (среза) парка техники из системы мониторинга Wialon, сделанные в разные моменты времени. Одна и та же машина (wialon_id) встречается в разных снапшотах с разными координатами и временными метками, что позволяет проследить перемещения.

Поле	Тип	Описание
wialon_id	bigint	Идентификатор единицы техники в Wialon (ключ)
nm	text	Название техники
cls, mu	integer	Служебные поля Wialon (можно игнорировать)
pos_t	bigint	Timestamp позиции (Unix epoch, секунды)
pos_y	double	Широта (Y-координата)

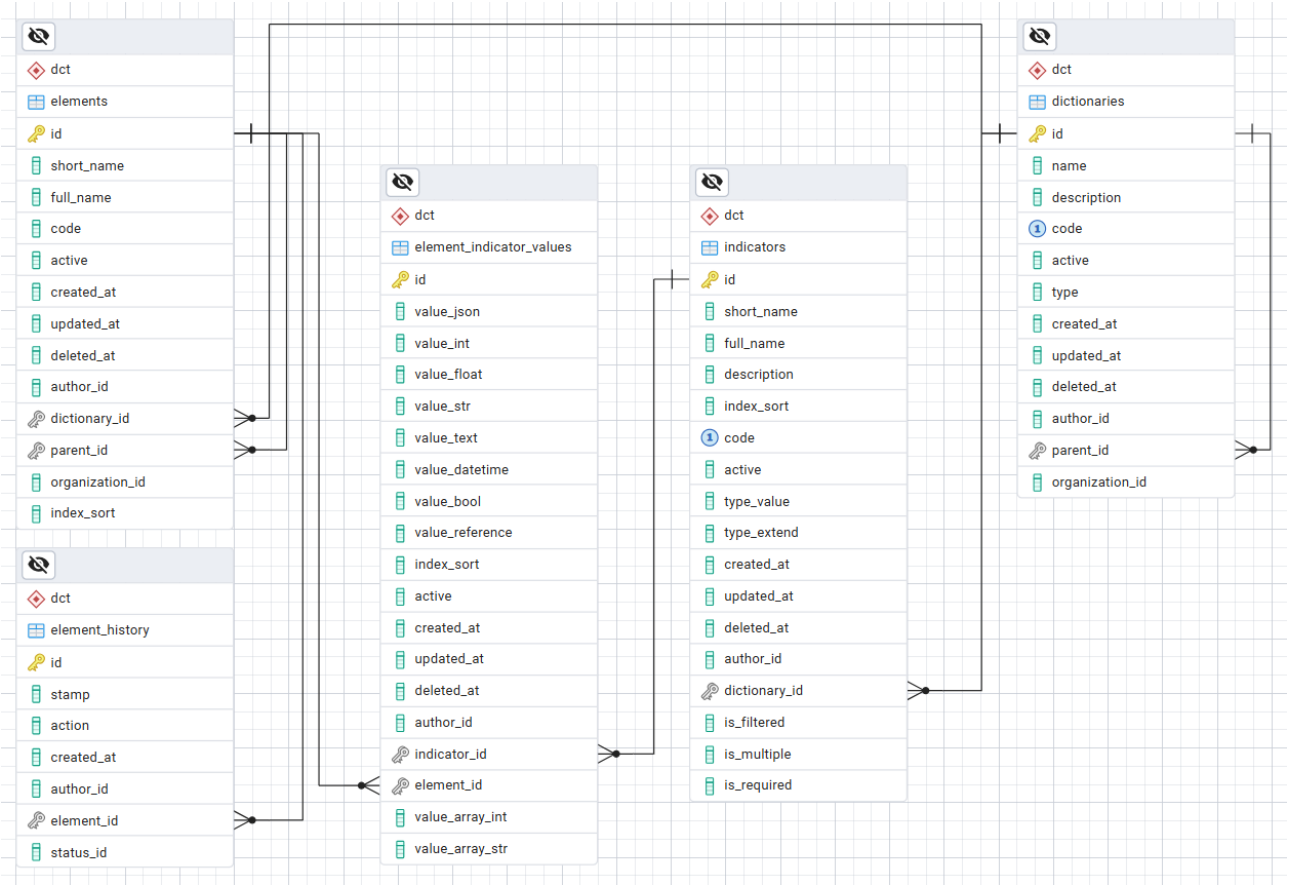
Поле	Тип	Описание
pos_x	double	Долгота (X-координата)
registration_plate	text	Госномер (обезличен)
payload_json	jsonb	Полный JSON снапшота Wialon (доп. данные)

Назначение:

- Текущие позиции техники (стартовые точки маршрутов): pos_x, pos_y из последнего снапшота.
- Вычисление средней скорости техники: Δ(расстояние) / Δ(pos_t) между снапшотами одного wialon_id.
- Исторический анализ (при необходимости): кластеры стоянок, паттерны движения, типовые маршруты.

4.5. Заявки (tasks)

Схема всех таблиц по заявкам:



Заявки на выполнение работ предоставляются в формате CSV и загружены в БД. Каждая заявка описывает задание, которое необходимо выполнить на конкретной скважине.

Поле	Тип	Описание
task_id	string	Уникальный идентификатор заявки
priority	enum	Приоритет: low / medium / high
planned_start	datetime	Плановое время начала работ
planned_duration_hours	float	Плановая длительность работ (часы)
destination_uwi	string	ID скважины (→ references.wells.uwi)
task_type	string	Тип работ (для проверки совместимости с техникой)
shift	enum	Смена: day / night

Поле	Тип	Описание
start_day	date	Дата начала выполнения

Вычисляемое поле: end_day = start_day + ceil(planned_duration_hours / часов_в_смене). Связь с БД: task.destination_uwi → references.wells.uwi → (longitude, latitude) → ближайший road_nodes.node_id.

Для SLA:
Веса приоритетов: high: 55%, medium: 35%, low: 10%.
Дедлайн начала работы по заявке по времени дня: high: +2, medium: +5, low: +12 часа.

Правила смен: Дневная смена: 08:00–20:00, Ночная смена: 20:00–08:00

4.6. Словарь совместимости

Словарь определяет, какая техника (или тип техники) может выполнять какой тип работ.

Использование: при назначении техники на заявку необходимо проверить, что тип техники допустим для task_type данной заявки. Это может быть реализовано как жёсткое ограничение (запрет назначения) или как мягкое (штраф за несовместимость).

5. Данные, необходимые для решения задачи

Вне зависимости от выбранного подхода для решения задачи маршрутизации необходимо подготовить следующие структуры данных из предоставленной БД. Если чего-то не хватает (сгенерировать по правдоподобным шаблонам).

5.1. Транспортные средства (vehicles)

Для каждой единицы техники необходимо определить стартовую позицию и параметры доступности.

Параметр	Источник	Статус	Комментарий
vehicle_id	wialon_units_snapshot*.wialon_id	**предоставляется**	Уникальные wialon_id из последнего снапшота
start_node	pos_x, pos_y → nearest road_nodes.node_id	**рассчитать**	KD-Tree поиск ближайшего узла графа к координатам техники
end_node	Бизнес-правило	**предоставляется**	Техника остаётся на последней точке (open-end маршрут)
start_time	pos_t + правило смены	**рассчитать**	Привязка к началу смены, перевод в минуты от начала горизонта
avg_speed	Найти в сети	**рассчитать**	Для конкретного типа техники
free_at	заявки	**рассчитать**	Данные о занятости техники. Машина свободна после конечного дня заявки.
skills	Словарь совместимости	**рассчитать**	Какие типы работ может выполнять данная техника

5.2. Заявки (jobs/orders)

Каждая заявка преобразуется в точку посещения с временным окном и длительностью обслуживания.

Параметр	Источник	Статус	Комментарий
task_id	CSV заявки	**предоставляется**	Идентификатор заявки
node_index	destination_uwi → wells → road_nodes	**рассчитать**	Цепочка: uwi → координаты скважины → ближайший узел графа

Параметр	Источник	Статус	Комментарий
tw_start, tw_end	start_day + shift → правило	рассчитать	Перевод даты и смены в абсолютные минуты на горизонте планирования
service_time	planned_duration_hours × 60	предоставляется	Длительность работ на объекте (в минутах)
priority → penalty	priority (low/med/high)	предоставляется	Участники задают маппинг приоритета в числовой штраф

5.3. Матрица расстояний и времён

Ключевой вычисляемый элемент: матрица стоимостей переезда между всеми парами точек задачи (стартовые позиции машин + точки заявок). Строится на основе графа дорог.

Параметр	Источник	Статус	Комментарий
Узлы графа	road_nodes.node_id, lon, lat	предоставляется	Вершины графа, готовы к использованию
Рёбра графа	road_edges.source, target, weight	предоставляется	Рёбра с весами, единицы указаны в метрах
Кратчайшие пути	Графовые алгоритмы	предоставляется	Вычислить для всех пар (start_nodes ∪ task_nodes)
time_matrix	distance / avg_speed	рассчитать	Матрица времён переезда, вычисляется из расстояний и скоростей

5.4. Точки назначения (destinations)

Параметр	Источник	Статус	Комментарий
uwi	wells.uwi	предоставляется	Идентификатор скважины, join с заявками
lon, lat	wells.longitude, wells.latitude	предоставляется	Координаты
nearest_node_id	wells → road_nodes (KDTree)	рассчитать	Map-matching: ближайший узел графа к каждой скважине

5.5. Временное измерение

Для учёта временных ограничений (смены, time windows, длительность работ) необходимо построить модель времени. Основные компоненты:

Параметр	Источник	Статус	Комментарий
time_callback	travel_time + service_time	рассчитать	Функция: время переезда + длительность работ на точке отправления
time windows	start_day + shift → правило	предоставляется	Правило перевода смен в часы предоставлено

6. Взаимосвязи данных и flow построения маршрута

6.1. Цепочка привязки данных

Основной flow от исходных данных к маршруту:

Заявка → скважина: task.destination_uwi → references.wells.uwi → (longitude, latitude).
Скважина → узел графа: wells(lon, lat) → nearest road_nodes.node_id = end_node.
Техника → узел графа: wialon_snapshot*(pos_x, pos_y) → nearest road_nodes.node_id = start_node.
Маршрут по графу: road_edges(source, target, weight) → кратчайший путь start_node → end_node.
Результат: длина маршрута = Σ weight рёбер; время = длина / avg_speed.

7. Что нужно реализовать

Ниже описаны программные модули, которые участникам необходимо спроектировать и реализовать для работы системы.

7.1. Модуль загрузки и индексации графа

Модуль при старте сервиса загружает таблицы road_nodes и road_edges из БД и строит in-memory граф. На этом же этапе формируется пространственный индекс (KD-Tree или аналог) по координатам узлов, чтобы любая произвольная точка (позиция техники, координаты скважины) могла быть привязана к ближайшему узлу графа (map-matching). Результат: объект графа, готовый для запуска алгоритмов кратчайшего пути, и функция snap_to_node(lon, lat) → node_id.

7.2. Сервис кратчайших путей и матрица стоимостей

На основе загруженного графа необходимо реализовать сервис, который по паре узлов (start_node, end_node) возвращает: длину кратчайшего пути (сумма weight), расчётное время в пути (длина / средняя скорость), упорядоченный список узлов маршрута и их координаты. Для пакетных запросов (например, при расчёте назначений на смену) модуль должен уметь формировать матрицу расстояний и времён для множества пар точек — стартовые позиции всех рассматриваемых машин × точки всех заявок. Выбор графового алгоритма остаётся за участниками; оценивается корректность результата и скорость работы на предоставленном графе.

7.3. Модуль состояния парка техники

Модуль отвечает за актуальную картину доступности парка. На входе — снапшоты из Wialon и текущий план заявок; на выходе — для каждой единицы техники: текущий статус (свободна / занята), прогнозируемое время и место освобождения (узел графа, где машина окажется после завершения текущей заявки), список совместимых типов работ (skills). Этот модуль не принимает решений о назначении — он поставляет данные для модулей скоринга и оптимизации. При изменении плана (новое назначение, отмена заявки) состояние парка должно пересчитываться.

7.4. Ядро оптимизации (решатель)

Центральный модуль системы, реализующий логику назначения техники на заявки. Принимает на вход матрицу стоимостей (7.2), состояние парка (7.3) и набор заявок с параметрами из раздела 4.5; возвращает план назначений с обоснованием каждого решения. Модуль должен поддерживать три режима работы: (а) одиночный запрос — для конкретной заявки выдать ранжированный список кандидатов с объяснением выбора; (б) пакетный запрос — для набора заявок на смену/день найти глобально оптимальное распределение техники, включая решение о том, какие заявки целесообразно объединить в multi-stop маршрут; (в) оценка группировки — для произвольного набора заявок рассчитать, какие из них выгодно объединить в один выезд, а какие обслуживать раздельно, с возвратом экономии относительно baseline. Подход к реализации выбирается участниками свободно (см. раздел 3.4). Приветствуется реализация наивного baseline-алгоритма (например, жадное назначение ближайшей свободной техники) для количественного сравнения с основным решением.

7.5. Функция скоринга и объяснимость

Каждая рекомендация, возвращаемая API, должна сопровождаться числовым score и текстовым обоснованием (reason). Участникам необходимо разработать формулу скоринга, которая агрегирует факторы, перечисленные в целевой функции (раздел 3.2), в единую метрику сравнения кандидатов. Формула должна быть документирована: какие веса присвоены расстоянию, времени ожидания, приоритету, штрафам за нарушение SLA и совместимости. Поле reason является опциональным и реализуется как бонусная функциональность: для его формирования рекомендуется использовать LLM, которая на основе числовых параметров выбора входа и выхода генерирует объяснение на естественном языке.

7.6. Визуализация маршрутов

API обязан возвращать геометрию маршрута в виде упорядоченного списка координат, пригодного для отрисовки polyline. Для демонстрации необходимо реализовать визуальное отображение маршрутов на карте (Leaflet, Folium, Kepler.gl или аналог). Визуализация должна показывать: стартовую позицию техники, точку назначения (скважину), линию маршрута по дорожному графу и, в случае multi-stop маршрута, промежуточные точки с порядком обхода.

8. Технические требования

8.1. Backend

- Язык: Python 3.11+.
- Framework: Django + DRF, FastAPI или Flask.
- REST API обязательно: JSON-запросы и ответы.

8.2. Минимальные эндпоинты API

POST /api/recommendations

Вход:

```
{
  "task_id": "T-2025-0042",
  "priority": "high",
  "destination_uwi": "05-1234-567",
  "planned_start": "2025-02-20T08:00:00",
  "duration_hours": 4.5
}
```

Выход (топ-3 units):

```
{
  "units": [
    {"wialon_id": 10234, "name": "АЦН-12 А045КМ", "eta_minutes": 38, "distance_km": 12.4, "score": 0.92, "reason": "ближайшая свободная, совместима по типу работ"},
    {"wialon_id": 10567, "name": "ЦА-320 В112ОР", "eta_minutes": 54, "distance_km": 18.7, "score": 0.78, "reason": "свободна, но дальше на 6 км"},
    {"wialon_id": 10891, "name": "АЦН-12 К330МН", "eta_minutes": 67, "distance_km": 14.1, "score": 0.65, "reason": "занята, освободится через 45 мин, затем ближайшая"}
  ]
}
```

POST /api/route

Вход:

```
{
  "from": {"wialon_id": 10234, "lon": 68.12345, "lat": 51.67890},
  "to": {"uwi": "05-1234-567", "lon": 68.09100, "lat": 51.70450}
}
```

Выход (По топ-3 units, в данном примере 1 unit**):**

```
{
  "distance_km": 12.4,
  "time_minutes": 38,
  "nodes": [4501, 4498, 4472, 4460, 4455, 4431, 4420],
  "coords": [
    [68.12345, 51.67890],
    [68.12100, 51.68012],
    [68.11450, 51.68500],
    [68.10800, 51.69100],
    [68.10200, 51.69600],
    [68.09500, 51.70100],
    [68.09100, 51.70450]
  ]
}
```

POST /api/multitask

Вход:

```
{
  "task_ids": ["T-2025-0042", "T-2025-0043", "T-2025-0044", "T-2025-0045"],
  "constraints": {
    "max_total_time_minutes": 480,
    "max_detour_ratio": 1.3
  }
}
```

Выход (пример 1):

```
{
  "groups": [
    ["T-2025-0042", "T-2025-0044"],
    ["T-2025-0043"],
    ["T-2025-0045"]
  ],
  "strategy_summary": "mixed",
  "total_distance_km": 41.2,
  "total_time_minutes": 195,
  "baseline_distance_km": 56.8,
  "baseline_time_minutes": 244,
  "savings_percent": 27.5,
  "reason": "заявки T-2025-0042 и T-2025-0044 в радиусе 3 км, объединение экономит 15.6 км; остальные заявки разнесены территориально, объединение превышает max_detour_ratio"
}
```

Выход (пример 2):

```
{
  "groups": [
    ["T-2025-0050"],
    ["T-2025-0051"],
    ["T-2025-0052"]
  ],
  "strategy_summary": "separate",
  "total_distance_km": 74.3,
  "total_time_minutes": 310,
  "baseline_distance_km": 74.3,
  "baseline_time_minutes": 310,
  "savings_percent": 0.0,
  "reason": "заявки расположены в разных частях месторождения (минимальное расстояние между парами — 28 км), объединение любой пары превышает max_detour_ratio 1.3; раздельное обслуживание оптимально"
}
```

Выход (пример 3):

```
{
  "groups": [
    ["T-2025-0060", "T-2025-0061", "T-2025-0062"]
  ],
  "strategy_summary": "single_unit",
  "total_distance_km": 19.8,
  "total_time_minutes": 142,
  "baseline_distance_km": 48.5,
  "baseline_time_minutes": 276,
  "savings_percent": 59.2,
  "reason": "все три заявки расположены в кусте скважин в радиусе 4 км, одна единица техники обходит все точки за один выезд с крюком менее 12%; экономия 28.7 км относительно раздельного обслуживания"
}
```

Сервис принимает список заявок, оценивает целесообразность объединения и возвращает оптимальную группировку: какие заявки выгодно объединить в multi-stop маршрут, а какие обслуживать отдельно. Поля baseline_distance_km и baseline_time_minutes содержат суммарные показатели при раздельном обслуживании каждой заявки, savings_percent — процент экономии предложенной группировки относительно baseline. Параметр max_detour_ratio ограничивает допустимое удлинение маршрута при объединении (например, 1.3 — не более 30% крюка), max_total_time_minutes — максимальное суммарное время выезда. Если объединение ни одной пары заявок не даёт выигрыша в рамках constraints, возвращается strategy_summary: "separate" с каждой заявкой в отдельной группе.

9. Критерии оценки

Критерий	Вес	Что оценивается
Качество алгоритма	**35%**	Логика выбора техники, корректность маршрутов по графу, способность находить неочевидные оптимизации
Техническое исполнение	**25%**	Качество кода, архитектура, работоспособность API, модульность и масштабируемость
Инновационность	**20%**	Использование AI/ML (если обосновано), оригинальность эвристик и подходов к решению
Практическая применимость	**15%**	Удобство для диспетчера, понятность обоснований, качество демо и сравнение с baseline
Работа над обработкой данных	**10%**	Качество преобразования входных данных

10. Ожидаемый результат

10.1. Работающий прототип

Backend + API, тестируемый через Postman/curl или простой фронтенд. Код выложен на GitHub/GitLab с README (как запустить, примеры запросов).

10.2. Демонстрация на 2–3 сценариях

- **Сценарий 1 (срочная заявка):** high priority → система выбирает оптимальную технику с обоснованием, показывает маршрут.
- **Сценарий 2 (сравнение):** medium priority → «просто ближайшая техника как baseline» vs улучшенный алгоритм → визуализация разницы в километраже/времени.
- **Сценарий 3 (многозадачность):** 3 заявки в одном районе → одна машина vs три отдельных, с обоснованием и цифрами.

10.3. Презентация (в слайдах)

Содержание: проблема и решение, архитектура, используемые алгоритмы и технологии, логика принятия решений (как учитываются факторы), демонстрация на реальных примерах, сравнение «наивный baseline» vs «оптимизированный подход» (таблица/график с цифрами), потенциал масштабирования.